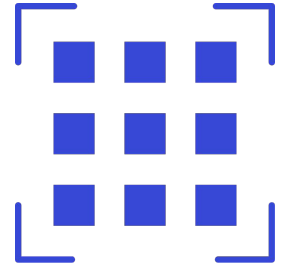
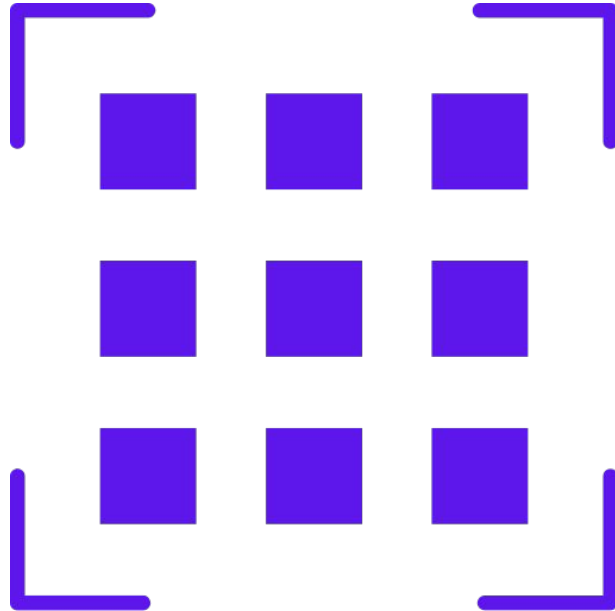


Array/Lists



Pre-requisites

- Basic Python Knowledge
- Time and Space Complexity
- Willingness to learn





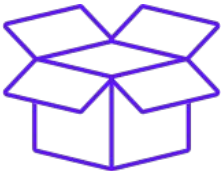
What are Data Structures?





Data Structures

- Particular way of **organizing**, and **storing** the data
- Tools to build efficient algorithms
- **The right one depends on the task**
- Common Data Structures:
 - Arrays
 - Linked Lists
 - Stacks
 - Queues
 - Trees
 - Maps(dictionaries)
 - Sets
 - Tries, etc...



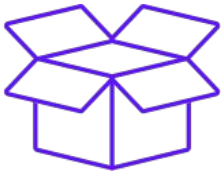
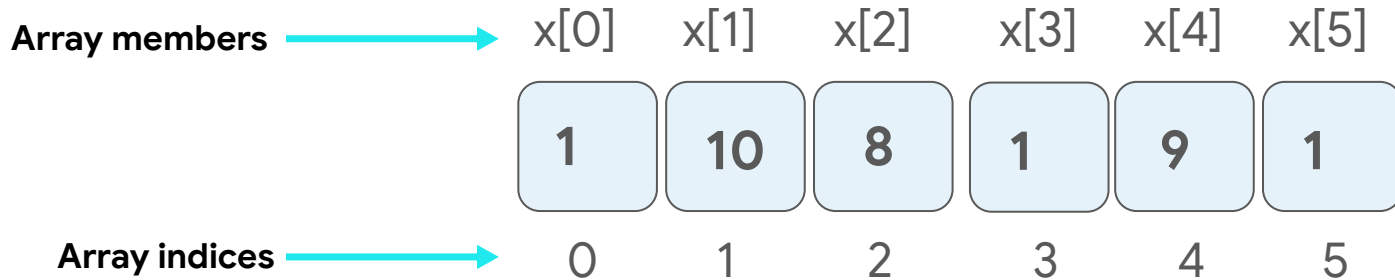
Arrays



Arrays



- Collection of items stored at **contiguous** memory locations.
- Storing multiple items of the **same type** together.
- Easier to calculate the position of each element by simply adding an offset to a base value.



Static Arrays

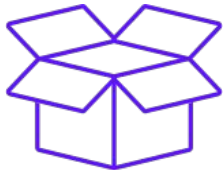




Static Arrays

- A type of array in which the size or length is determined when the array is created and/or allocated.

	7	3	6	
	*0	*4	*8	



Static Arrays



Value

Address

	1	3	5	
	*0	*4	*8	

Value

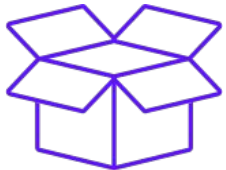
Address

	a	b	c	
	*0	*1	*2	

Static Arrays - Reading

- Reading is Very fast : $O(1)$
- Why?

```
print(my_array[4])
```

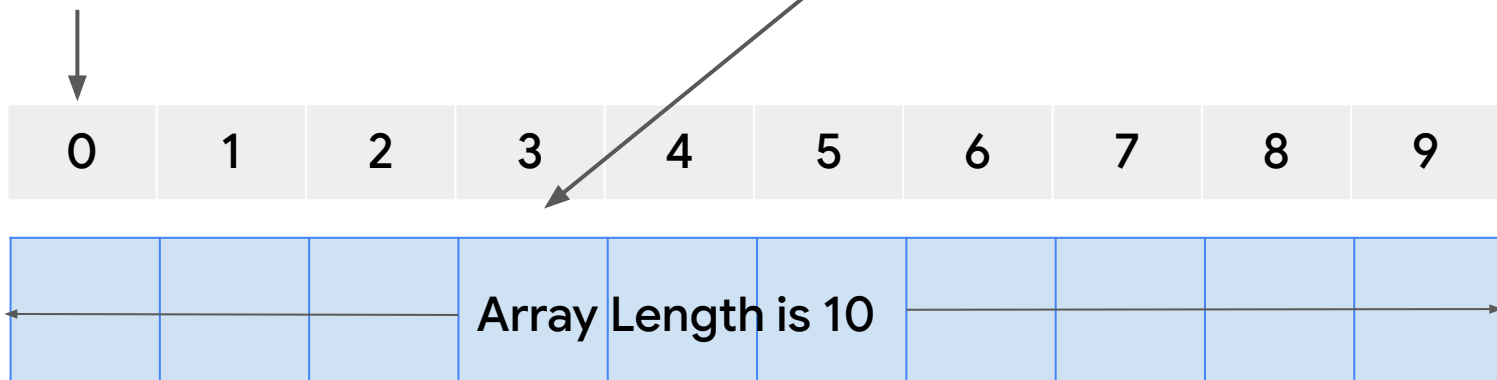




Static Arrays - Reading

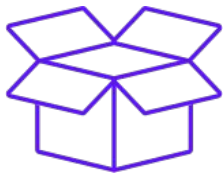
First Index

4th element



Each integer occupies 4 bytes, so let us say the 0th index is at memory location 1400

$$1400 + (3 * 4) = 1412$$



Static Arrays - Writing

- Writing is an instant operation



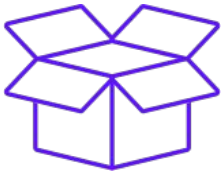
5

Value

Address

	1	3	0	
	*0	*4	*8	

5



Static Arrays - Writing

- Overwriting is an instant operation



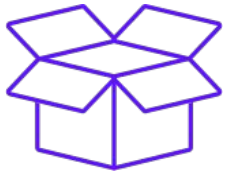
4

Value

Address

	1	3	5	
	*0	*4	*8	

4





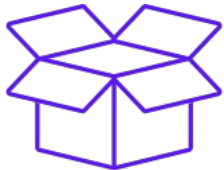
Static Arrays - Writing

- What happens when we want to add an element to our array but don't have any empty spots?



7

Value	1	3	5	
Address	*0	*4	*8	





Static Arrays - Writing

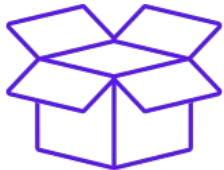
- The operating system might be using this part of the memory for some other purpose.
- So **we are not necessarily allowed** to put our seven there.



Value

Address

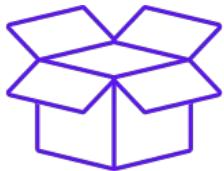
	1	3	5	
	*0	*4	*8	





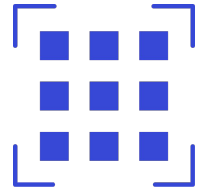
Static Arrays - Summary

Operation	Big-O Time
Read i-th Element	$O(1)$
Write i-th Element	$O(1)$



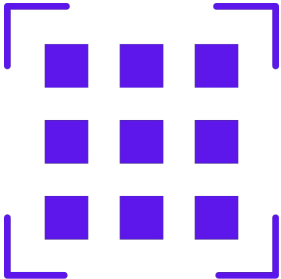
Dynamic Arrays





Dynamic Arrays

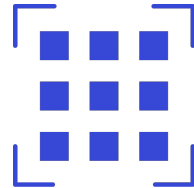
- A Dynamic array has the ability to **resize** itself automatically when an element is inserted or deleted.
- **Double the size** when it reaches capacity, **cut by half** when it's down to quarter





**Does Python list allow the storage
of different types ?**



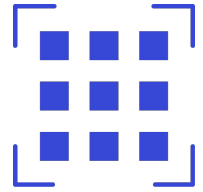


Dynamic Arrays

- No way to find out `arr[3]` without traversing the full list

```
arr = [3, 0.5, 'a', "1"]
```

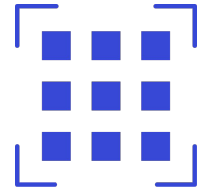
Value	3	0.5	a	"1"
Address	*0	*4	*12	*13



Dynamic Arrays

- A list in Python is implemented as **an array of references**.
- what you are actually creating is an array of references like :

0 1 2 3
[0xa3d25342, 0x635423fa, 0xff243546, 0x2545fade]

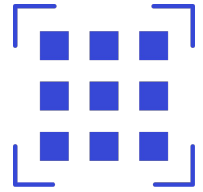


Dynamic Arrays

- Each element is a reference that "points" to the respective objects in memory

	*0	*1	*2	*3	
Reference	*10	*14	*22	*23	
Value	3	0.5	a	"1"	

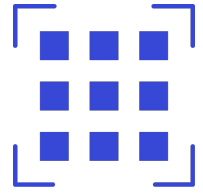
- Memory Address:** A unique identifier for a specific location in memory where data is stored, typically represented as a hexadecimal value.
- Reference:** A variable that acts as an alias for another object, allowing indirect access to that object without directly storing its memory address.



Dynamic Arrays

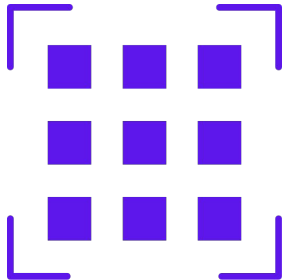
- This works because the **all references are of the same size** unlike the actual values they point to.

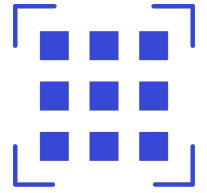
	*0	*1	*2	*3	
Reference	*10	*14	*22	*23	
Value	3	0.5	a	"1"	



Common Terminologies

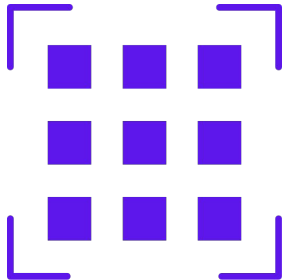
- **Pushing/Appending** : inserting an element at the next empty spot
- **Popping**: removing the element from the end of the list





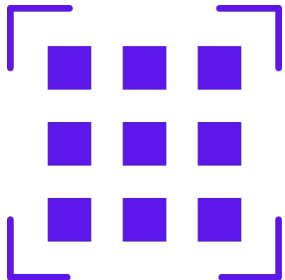
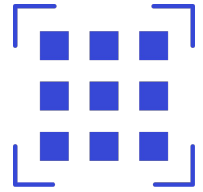
Pushing

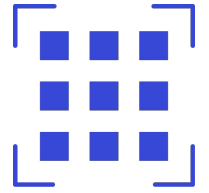
- Whenever we reach our array's capacity, our interpreter allocates another memory which is **double our original capacity** and **copies over all the old elements with now more space**.
- Isn't this operation costly?



Pushing

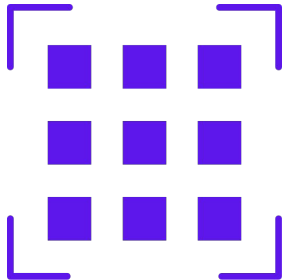
- It takes **amortized $O(1)$ time**! Let's see why

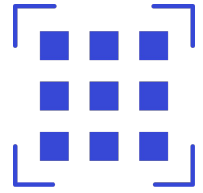




Pushing

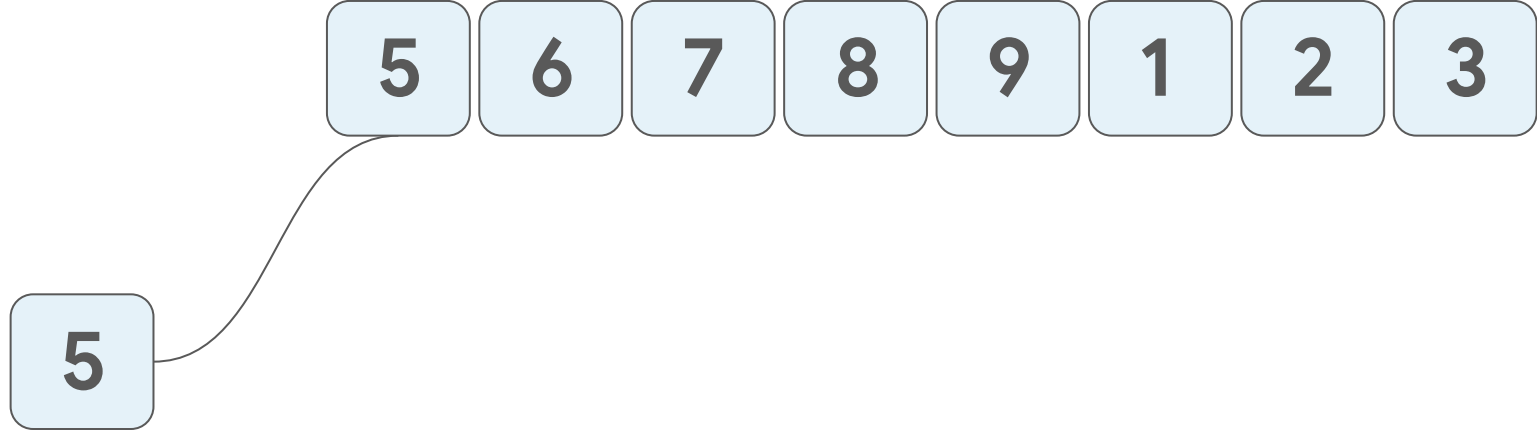
- Our goal is to push 8 elements to a dynamic array.
- **Assumption: Our initial capacity is of size one.**



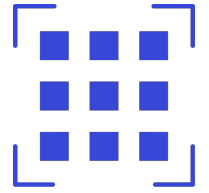


Pushing

- After pushing 5, no place for 6

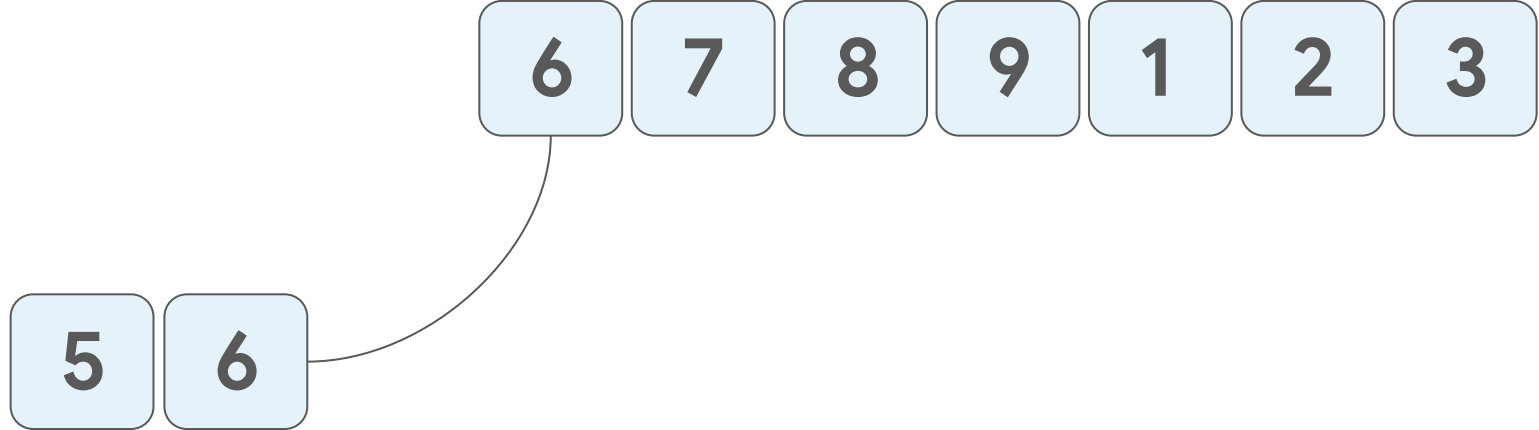


Operation count: 1

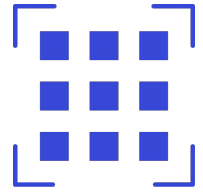


Pushing

- Copy the original array ([5]) to the new allocated memory which is double the old size and push 6 onto it

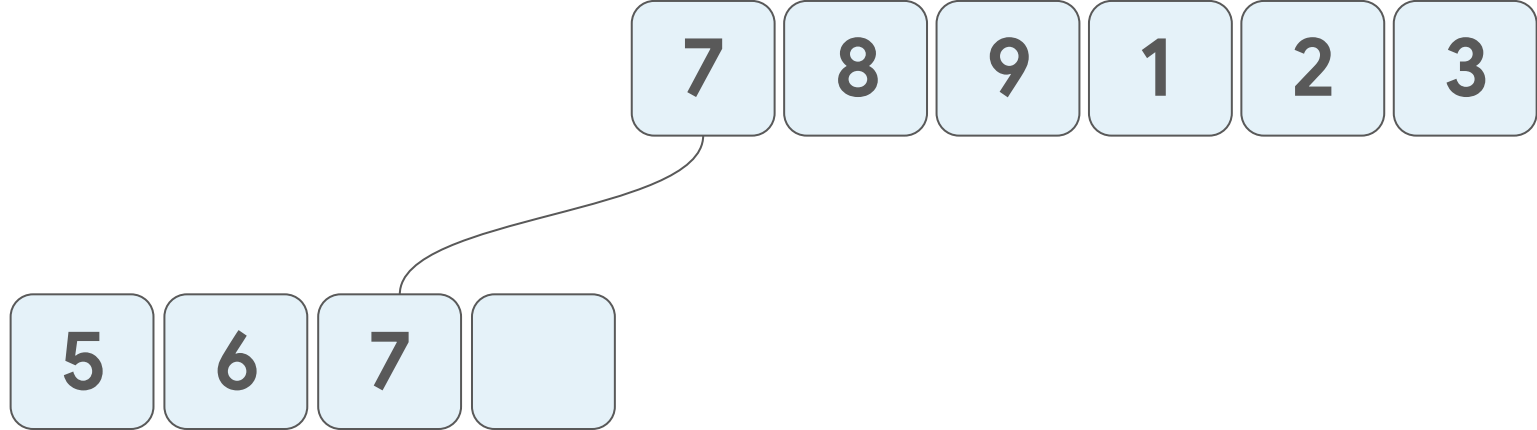


Operation count: 1 + 2

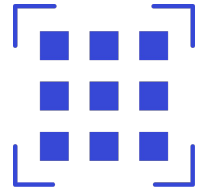


Pushing

- Allocate a new array of size 4 and copy over all the old elements and then we append 7

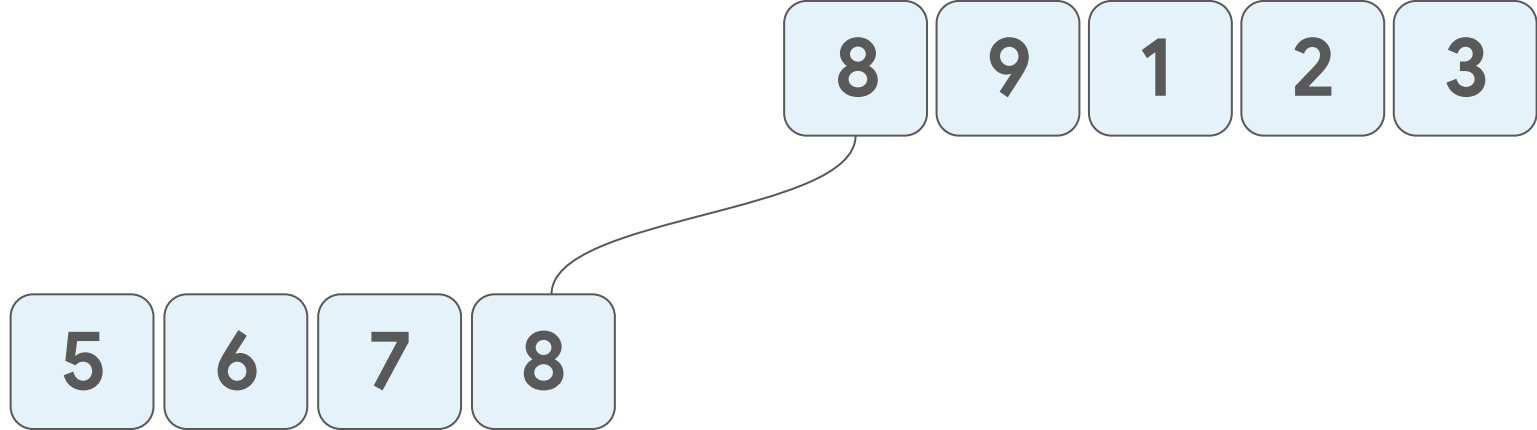


Operation count: $1 + 2 + 3$

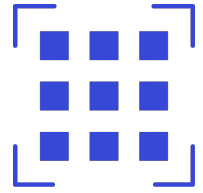


Pushing

- Push 8 onto the array. Now we have run out of space for the next elements.

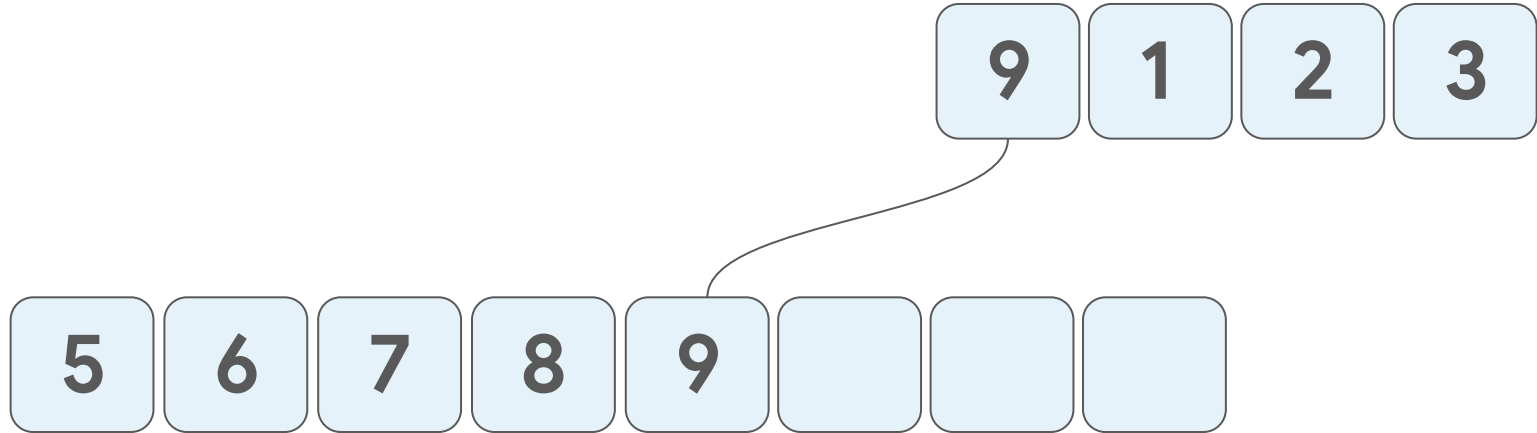


Operation count: $1 + 2 + 4$

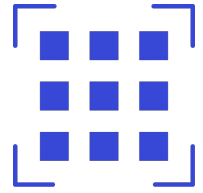


Pushing

- Allocate a new array of size 8 and copy over all the old elements and then we append the element 9

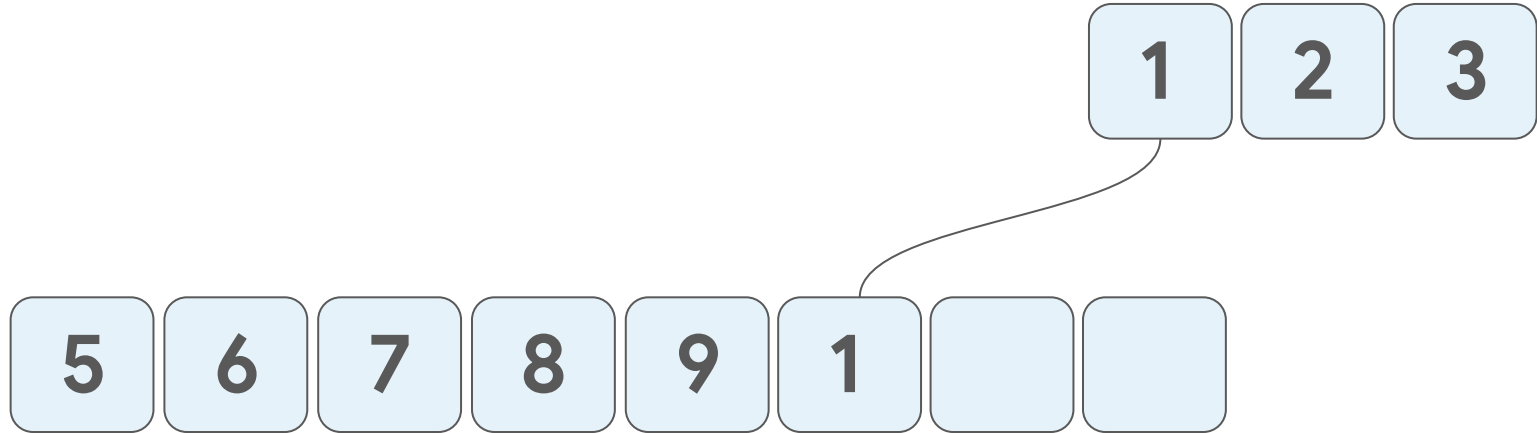


Operation count: $1 + 2 + 4 + 5$

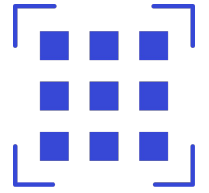


Pushing

- Push/append 1 into the array

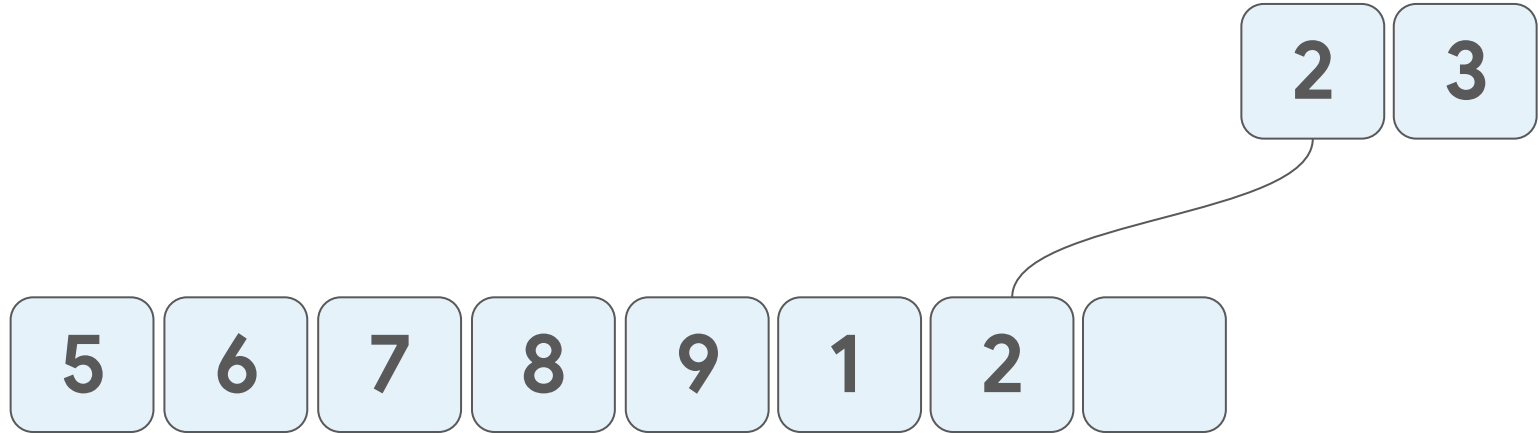


Operation count: $1 + 2 + 4 + 6$

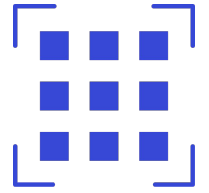


Pushing

- Push/append 2 into the array

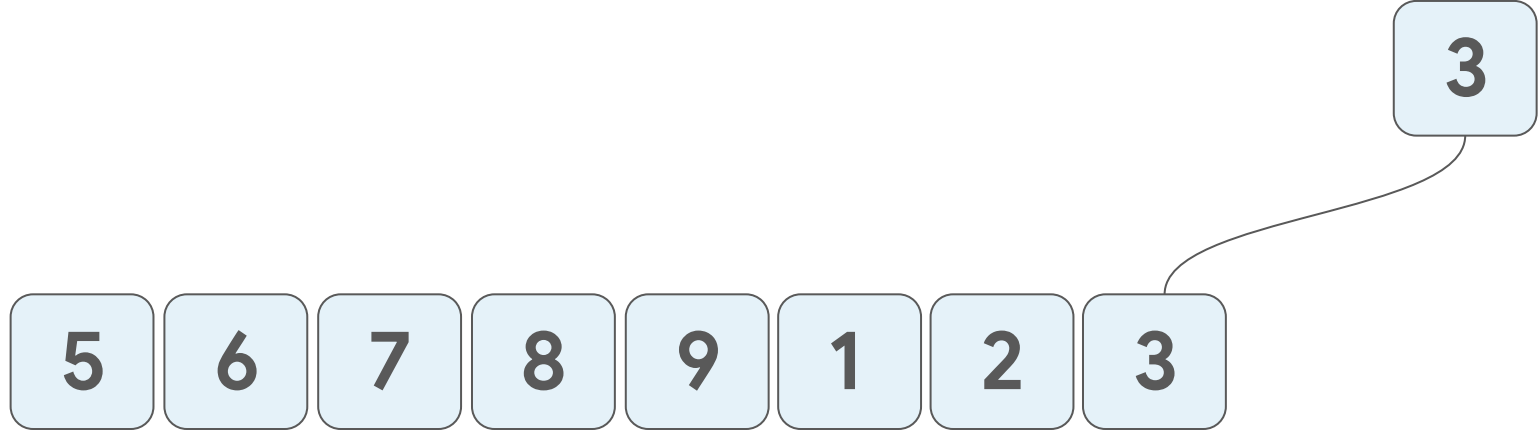


Operation count: $1 + 2 + 4 + 7$

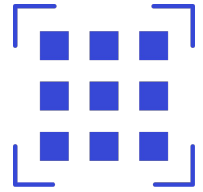


Pushing

- Push/append 3 into the array



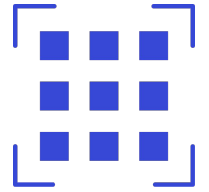
Operation count: $1 + 2 + 4 + 8$



Pushing- Power Series

- For a length of 8, it took us $1 + 2 + 4 + 8$ operations.
- The last summand always dominates the sum.

$$\begin{aligned} & \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \boxed{} \\ = & \boxed{} \boxed{} \boxed{} \boxed{} + \boxed{} \boxed{} \boxed{} \boxed{} \\ = & \boxed{} \boxed{} \boxed{} \boxed{} + \boxed{} \boxed{} + \boxed{} \boxed{} \\ \geq & \boxed{} \boxed{} \boxed{} \boxed{} + \boxed{} \boxed{} + \boxed{} + \boxed{} \end{aligned}$$



Pushing - Power Series

- For a length of 8, it took us $1 + 2 + 4 + 8$ operations.
- The last summand always dominates the sum.

$$2 \geq 1$$

$$4 \geq 1 + 2$$

$$8 \geq 1 + 2 + 4$$

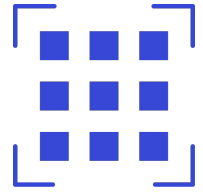
.

.

.

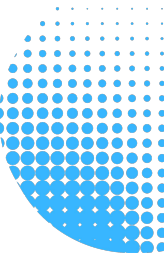
$$2^*N \geq 1 + 2 + 4 + 8 + \dots + N$$

- Thus to push N elements, it will take us no more than $2N$ operations, making the time complexity $O(N)$.



Pushing - Power Series

- This means, to push a single element onto an array, on average, we say it is an instant - $O(1)$ - operation.
- Because after a while, the copying operation becomes very **infrequent**.
- Hence, pushing an element onto an array is **Amortized $O(1)$** operation.



Dynamic Arrays - Insertion

- Inserting an element in an arbitrary position is, in the worst case, an $O(n)$ operation.



Value

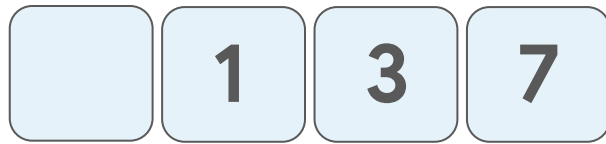
Address

		1	3	7
	*0	*4	*8	*12



Dynamic Arrays - Insertion

- We need to **shift** all elements to the right of the new element, to not lose data, and create the empty space needed.



↑
5

Value

Address

		1	3	7
	*0	*4	*8	*12



Dynamic Arrays - Insertion

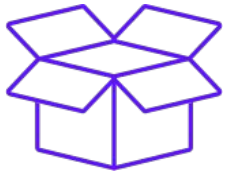
- The shifting is what costs us time.



Value

Address

	5	1	3	7
	*0	*4	*8	*12



Dynamic Arrays - Removal

- What about removing an element?



Value

Address

	5	1	3	7
	*0	*4	*8	*12

Dynamic Arrays - Removal

- It is $O(n)$ at worst.

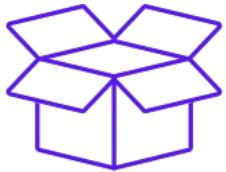


Value

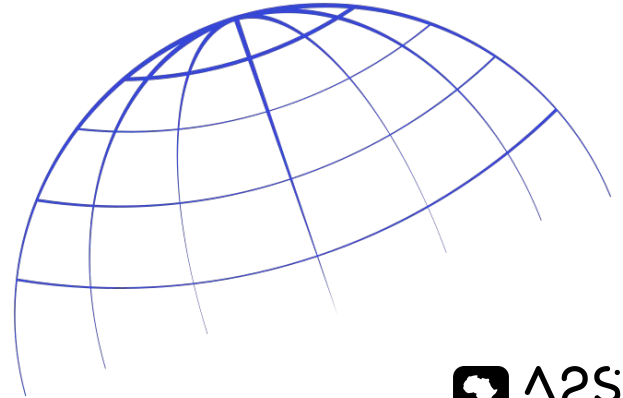
Address

	1	3	7	
	*0	*4	*8	

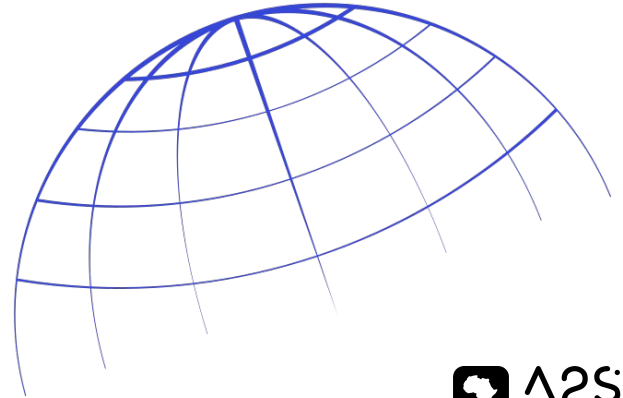
The diagram illustrates the removal process in a dynamic array. It shows a memory layout with four columns. The first three columns contain the values 1, 3, and 7. The fourth column is empty. Below the values, the addresses *0, *4, and *8 are shown. Three curved arrows above the table indicate the shifting of elements: the first arrow points from the second column to the first, the second from the third to the second, and the third from the fourth to the third, representing the shift of elements to the right to fill the gap.

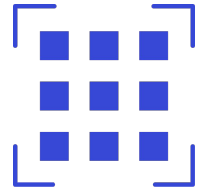


Checkpoint



Traversing Lists





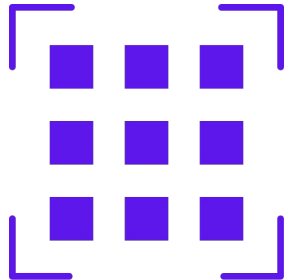
Traversing Lists - Reverse Traversal

Approach

- Start from the last index
- Decrement by 1 until -1

Implementation

```
for index in range(len(array)-1, -1, -1):  
    print(array[index])
```



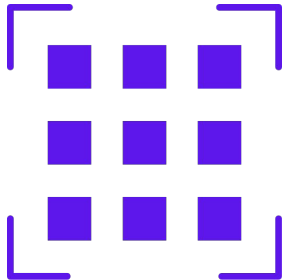
Traversing Lists - Even/Odd Indices Traversal

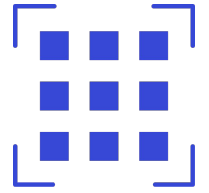
Approach

- Start from the first even/odd index
- Increment by 2

Implementation

```
step = 2 # incremental value
# even indices traversal
size = len(array)
for index in range(0, size, step):
    print(array[index])
```





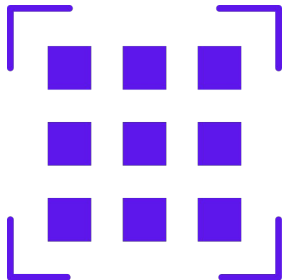
Traversing Lists - Circular Traversal

Approach

- Start from first index to $\text{size} \times 2$
- Incrementally go 1 step but modulo index by size

Implementation

```
size = len(array)
for index in range(0, 2 * size):
    value = array[index % size]
    print(value)
```

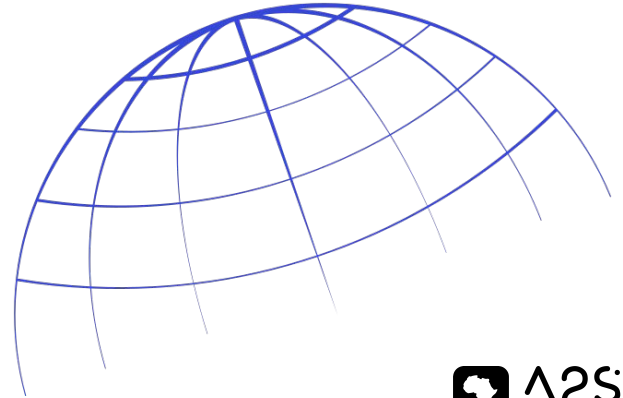


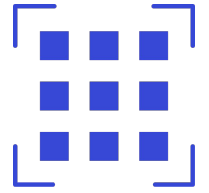
Pair Programming

Find the winner of the circular game



Swapping Elements





Swapping Elements

Approach

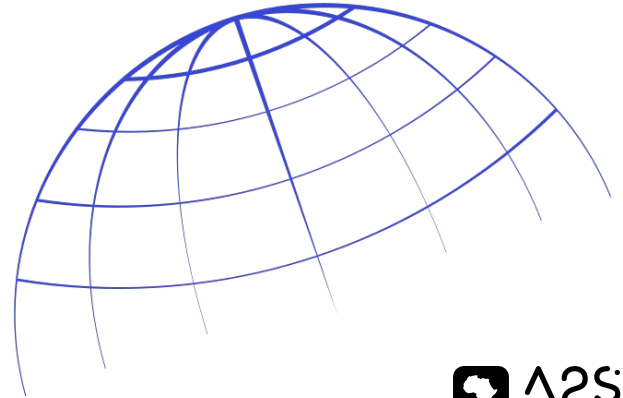
- Approach 1
 - Store first element in a variable
 - Change first element with second element
 - Change second element with stored value
- Approach 2 : Use **tuple unpacking** to swap

Implementation

```
# approach 1
temp = arr[i]
arr[i] = arr[j]
arr[j] = temp
```

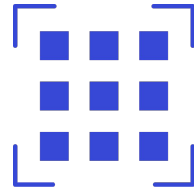
```
# approach 2
arr[i], arr[j] = arr[j], arr[i]
```

Storing letter and Numbers



Storing - Using Arrays as Dictionaries

- For Letters
 - Character's ASCII value can be the index by correcting the offset by 65 ('A') for uppercase letters and 97 ('a') for lowercase letters
- For Numbers
 - The number itself is the key
- Note: the advantage of this approach is that you can use built in functions like `sum`, `sort`, `max`, `min` on the frequency list whereas maps don't give you this flexibilities.



Storing - Implementation

```
word = "abceabce"
arrayDictionary = [0]*26
offset = ord('a') # 'a' has ASCII value of 97
for char in word:
    ascii = ord(char)
    arrayDictionary[ascii - offset] += 1

print(arrayDictionary)
# [2, 2, 2, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
# a b c d e f g h i j k l m n, o p q r s t u v w x y z
```

Pair Programming

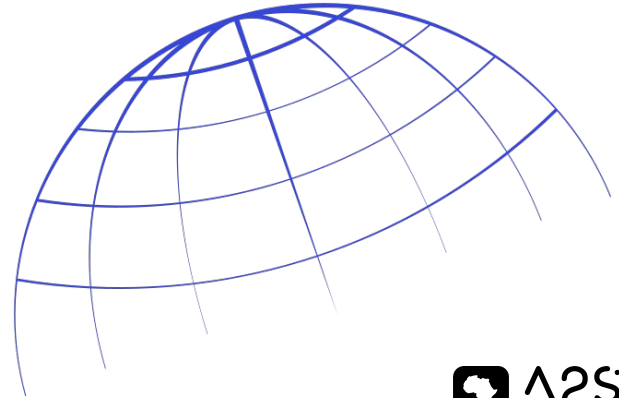
Reverse string

Apply operations to an array



Resources

- <https://neetcode.io/courses/dsa-for-beginners/0>: useful on laying out the fundamentals
- <https://www.geeksforgeeks.org/python-arrays/>: covers large range of topics for python
- [Python List pop\(\) Method - GeeksforGeeks](#): good examples
- [Python List insert\(\) Method With Examples - GeeksforGeeks](#): good examples
- [Python | Which is faster to initialize lists? - GeeksforGeeks](#): good article on list initialization



Quote of the Day

If you want to enjoy the rainbow,
be prepared to endure the storm

Warren W. Wiersbe

